

LAB EXPERIMENT

NAME : B Hitesh Roy

REG NO : 1923272036

COURSE CODE: (CSA0618)

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHM.

Experiment 1: Linear Search (Iterative)

Aim

To analyze the time complexity of linear search.

Algorithm

1. Read the array and the key element.
2. Traverse the array from the first element.
3. Compare each element with the key.
4. If the key is found, display its index.
5. Otherwise, display "Key not found."

Program

```
def linear_search(arr, key):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == key:
```

```
            return i
```

```
    return -1
```

```
arr = [10, 25, 30, 45, 50]
```

```
key = 30
```

```
result = linear_search(arr, key)
```

```
if result != -1:
```

```
    print("Key found at index", result)
```

```
else:
```

```
    print("Key not found")
```

Sample Input

Array = [10, 25, 30, 45, 50]

Key = 30

Sample Output

Key found at index 2

Result

Thus, the linear search algorithm was implemented successfully and the time complexity was analyzed.

Experiment 2: Binary Search (Recursive)

Aim

To compare the recursive binary search complexity.

Algorithm

1. Read the sorted array and key.
2. Find the middle element.
3. If the key matches the middle element, return its index.
4. If the key is smaller, search the left half recursively.
5. Otherwise, search the right half recursively.
6. If the key is not found, return -1.

Program

```
def binary_search(arr, low, high, key):
```

```
if low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        return mid

    elif arr[mid] > key:
        return binary_search(arr, low, mid - 1, key)

    else:
        return binary_search(arr, mid + 1, high, key)

return -1
```

```
arr = [5, 10, 15, 20, 25]
```

```
key = 20
```

```
result = binary_search(arr, 0, len(arr)-1, key)
```

```
if result != -1:
```

```
    print("Key found at index", result)
```

```
else:
```

```
    print("Key not found")
```

Sample Input

```
Array = [5, 10, 15, 20, 25]
```

```
Key = 20
```

Sample Output

```
Key found at index 3
```

Result

Thus, the recursive binary search algorithm was implemented successfully and its logarithmic complexity was verified.

Experiment 3: Factorial (Iterative and Recursive)

Aim

To compare iterative and recursive factorial computation.

Algorithm

1. Read the value of n.
2. Calculate factorial using an iterative method.
3. Calculate factorial using a recursive method.
4. Display both results.

Program

```
def factorial_iterative(n):  
    fact = 1  
    for i in range(1, n + 1):  
        fact *= i  
    return fact  
  
def factorial_recursive(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial_recursive(n - 1)  
  
n = 5  
  
print("Iterative:", factorial_iterative(n))
```

```
print("Recursive:", factorial_recursive(n))
```

Sample Input

n = 5

Sample Output

Iterative: 120

Recursive: 120

Result

Thus, factorial was computed successfully using both iterative and recursive methods.

Experiment 4: Fibonacci Series

Aim

To analyze recursive and iterative Fibonacci generation.

Algorithm

1. Read the value of n.
2. Generate Fibonacci numbers iteratively.
3. Generate Fibonacci numbers recursively using memoization.
4. Display the series.

Program

```
def fibonacci_iterative(n):
```

```
    series = []
```

```
    a, b = 0, 1
```

```
    for _ in range(n):
```

```
        series.append(a)
```

```
        a, b = b, a + b
```

```
return series
```

```
def fibonacci_recursive(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = fibonacci_recursive(n-1, memo) + fibonacci_recursive(n-2, memo)  
    return memo[n]
```

```
def recursive_series(n):  
    return [fibonacci_recursive(i) for i in range(n)]
```

```
n = 6
```

```
print("Iterative:", fibonacci_iterative(n))
```

```
print("Recursive:", recursive_series(n))
```

Sample Input

```
n = 6
```

Sample Output

```
Iterative: [0, 1, 1, 2, 3, 5]
```

```
Recursive: [0, 1, 1, 2, 3, 5]
```

Result

Thus, the Fibonacci series was generated successfully using both iterative and recursive methods.

Experiment 5: Matrix Multiplication

Aim

To analyze the complexity of matrix multiplication using nested loops.

Algorithm

1. Read two matrices.
2. Initialize the result matrix with zeros.
3. Multiply rows and columns using three nested loops.
4. Store the sum of products.
5. Display the resultant matrix.

Program

```
A = [[1, 2], [3, 4]]
```

```
B = [[5, 6], [7, 8]]
```

```
result = [[0, 0], [0, 0]]
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        for k in range(2):
```

```
            result[i][j] += A[i][k] * B[k][j]
```

```
print(result)
```

Sample Input

```
A = [[1,2],[3,4]]
```

```
B = [[5,6],[7,8]]
```

Sample Output

```
[[19, 22], [43, 50]]
```

Result

Thus, matrix multiplication was implemented successfully using nested loops and the output matrix was obtained correctly.

Experiment 6: Merge Sort

Aim

To analyze the divide-and-conquer sorting technique using Merge Sort.

Algorithm

1. Divide the array into two halves.
2. Recursively sort each half.
3. Merge the sorted halves into a single sorted array.
4. Display the sorted array.

Program

```
def merge_sort(arr):  
    if len(arr) > 1:  
        mid = len(arr) // 2  
        left = arr[:mid]  
        right = arr[mid:]  
  
        merge_sort(left)  
        merge_sort(right)  
  
        i = j = k = 0  
  
        while i < len(left) and j < len(right):  
            if left[i] < right[j]:  
                arr[k] = left[i]  
                i += 1
```



```
else:
```

```
    arr[k] = right[j]
```

```
    j += 1
```

```
    k += 1
```

```
while i < len(left):
```

```
    arr[k] = left[i]
```

```
    i += 1
```

```
    k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

```
    k += 1
```

```
arr = [38, 27, 43, 3, 9, 82, 10]
```

```
merge_sort(arr)
```

```
print(arr)
```

Sample Input

```
[38, 27, 43, 3, 9, 82, 10]
```

Sample Output

```
[3, 9, 10, 27, 38, 43, 82]
```

Result

Thus, Merge Sort was implemented successfully and the array was sorted correctly.

Experiment 7: Quick Sort

Aim

To analyze recursive Quick Sort using divide-and-conquer.

Algorithm

1. Select the last element as the pivot.
2. Partition the array around the pivot.
3. Recursively sort the left and right partitions.
4. Display the sorted array.

Program

```
def partition(arr, low, high):  
    pivot = arr[high]  
    i = low - 1  
  
    for j in range(low, high):  
        if arr[j] <= pivot:  
            i += 1  
            arr[i], arr[j] = arr[j], arr[i]  
  
    arr[i + 1], arr[high] = arr[high], arr[i + 1]  
    return i + 1  
  
def quick_sort(arr, low, high):  
    if low < high:  
        pi = partition(arr, low, high)
```

```
    quick_sort(arr, low, pi - 1)
    quick_sort(arr, pi + 1, high)
```

```
arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr, 0, len(arr) - 1)
print(arr)
```

Sample Input

[10, 7, 8, 9, 1, 5]

Sample Output

[1, 5, 7, 8, 9, 10]

Result

Thus, Quick Sort was implemented successfully and the array was sorted correctly.

Experiment 8: Tower of Hanoi

Aim

To analyze recursive implementation of the Tower of Hanoi problem.

Algorithm

1. If only one disk exists, move it from source to destination.
2. Move $n-1$ disks from source to auxiliary.
3. Move the largest disk to destination.
4. Move $n-1$ disks from auxiliary to destination.

Program

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
    return
```

```
tower_of_hanoi(n - 1, source, destination, auxiliary)

print("Move disk", n, "from", source, "to", destination)

tower_of_hanoi(n - 1, auxiliary, source, destination)
```

```
tower_of_hanoi(3, 'A', 'B', 'C')
```

Sample Input

n = 3

Sample Output

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

Result

Thus, the Tower of Hanoi problem was solved successfully using recursion.

Experiment 9: GCD (Euclidean Algorithm)

Aim

To implement the Euclidean Algorithm for finding the Greatest Common Divisor (GCD).

Algorithm

1. Read two integers.
2. If the second number is zero, return the first number.
3. Otherwise, recursively compute GCD of the second number and the remainder.

4. Display the GCD.

Program

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)
```

```
a = 48
```

```
b = 18
```

```
print("GCD =", gcd(a, b))
```

Sample Input

```
a = 48
```

```
b = 18
```

Sample Output

```
GCD = 6
```

Result

Thus, the GCD was found successfully using the Euclidean Algorithm.

Experiment 10: Insertion Sort

Aim

To analyze the performance of the Insertion Sort algorithm.

Algorithm

1. Start from the second element.
2. Compare it with previous elements.
3. Shift larger elements one position to the right.

4. Insert the current element in its correct position.
5. Repeat until the array is sorted.

Program

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
  
        while j >= 0 and arr[j] > key:  
            arr[j + 1] = arr[j]  
            j -= 1  
  
        arr[j + 1] = key
```

```
arr = [12, 11, 13, 5, 6]  
insertion_sort(arr)  
print(arr)
```

Sample Input

[12, 11, 13, 5, 6]

Sample Output

[5, 6, 11, 12, 13]

Result

Thus, the Insertion Sort algorithm was implemented successfully and the array was sorted correctly.

Experiment 11: Heap Sort

Aim

To analyze heap-based sorting using the Heap Sort algorithm.

Algorithm

1. Build a max heap from the given array.
2. Swap the root element with the last element.
3. Reduce the heap size by one.
4. Heapify the root again.
5. Repeat until the array is sorted.

Program

```
def heapify(arr, n, i):  
    largest = i  
    left = 2 * i + 1  
    right = 2 * i + 2  
  
    if left < n and arr[left] > arr[largest]:  
        largest = left  
  
    if right < n and arr[right] > arr[largest]:  
        largest = right  
  
    if largest != i:  
        arr[i], arr[largest] = arr[largest], arr[i]  
        heapify(arr, n, largest)  
  
def heap_sort(arr):
```

```
n = len(arr)

for i in range(n // 2 - 1, -1, -1):
    heapify(arr, n, i)

for i in range(n - 1, 0, -1):
    arr[i], arr[0] = arr[0], arr[i]
    heapify(arr, i, 0)

arr = [4, 10, 3, 5, 1]
heap_sort(arr)
print(arr)
```

Sample Input

[4, 10, 3, 5, 1]

Sample Output

[1, 3, 4, 5, 10]

Result

Thus, the Heap Sort algorithm was implemented successfully and the array was sorted correctly.

Experiment 12: Counting Inversions**Aim**

To count the number of inversions in an array using a modified Merge Sort.

Algorithm

1. Divide the array into two halves.
2. Recursively count inversions in both halves.
3. Count inversions while merging the two sorted halves.

4. Display the total inversion count.

Program

```
def merge_count(arr):  
    if len(arr) <= 1:  
        return arr, 0  
  
    mid = len(arr) // 2  
    left, inv_left = merge_count(arr[:mid])  
    right, inv_right = merge_count(arr[mid:])  
  
    merged = []  
    i = j = inv = 0  
  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            merged.append(left[i])  
            i += 1  
        else:  
            merged.append(right[j])  
            inv += len(left) - i  
            j += 1  
  
    merged.extend(left[i:])  
    merged.extend(right[j:])  
  
    return merged, inv + inv_left + inv_right
```

```
arr = [2, 4, 1, 3, 5]
_, count = merge_count(arr)
print("Inversions =", count)
```

Sample Input

[2, 4, 1, 3, 5]

Sample Output

Inversions = 3

Result

Thus, the inversion count was found successfully using the modified Merge Sort algorithm.

Experiment 13: Maximum Subarray Sum (Kadane's Algorithm)**Aim**

To find the maximum subarray sum using Kadane's Algorithm.

Algorithm

1. Initialize current sum and maximum sum.
2. Traverse the array.
3. Update the current sum.
4. Update the maximum sum whenever a larger value is found.
5. Display the maximum subarray sum.

Program

```
def kadane(arr):
    max_sum = arr[0]
    current_sum = arr[0]

    for i in range(1, len(arr)):
```

```
current_sum = max(arr[i], current_sum + arr[i])
```

```
max_sum = max(max_sum, current_sum)
```

```
return max_sum
```

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
```

```
print("Maximum Subarray Sum =", kadane(arr))
```

Sample Input

```
[-2, -3, 4, -1, -2, 1, 5, -3]
```

Sample Output

Maximum Subarray Sum = 7

Result

Thus, the maximum subarray sum was found successfully using Kadane's Algorithm.

Experiment 14: Exponentiation (Fast Power)

Aim

To compare iterative and recursive fast exponentiation methods.

Algorithm

1. Read the values of x and n.
2. Compute x^n iteratively.
3. Compute x^n recursively using fast exponentiation.
4. Display both results.

Program

```
def power_iterative(x, n):
```

```
    result = 1
```

```
    for _ in range(n):
```

```
        result *= x

    return result


def power_recursive(x, n):

    if n == 0:

        return 1

    if n % 2 == 0:

        half = power_recursive(x, n // 2)

        return half * half

    return x * power_recursive(x, n - 1)


x = 2

n = 10


print("Iterative:", power_iterative(x, n))

print("Recursive:", power_recursive(x, n))
```

Sample Input

```
x = 2

n = 10
```

Sample Output

```
Iterative: 1024

Recursive: 1024
```

Result

Thus, exponentiation was implemented successfully using both iterative and recursive methods.

Experiment 15: Closest Pair of Points (Brute Force)

Aim

To find the closest pair of points in a 2D plane using the brute-force method.

Algorithm

1. Read the list of points.
2. Compute the distance between every pair of points.
3. Store the minimum distance and the corresponding pair.
4. Display the closest pair and the minimum distance.

Program

```
import math

def closest_pair(points):
    min_dist = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt((points[i][0] - points[j][0]) ** 2 +
                             (points[i][1] - points[j][1]) ** 2)

            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])

    return pair, min_dist
```

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

```
pair, distance = closest_pair(points)
```

```
print("Closest Pair:", pair)
```

```
print("Distance =", distance)
```

Sample Input

```
[(1,2), (4,5), (7,8), (3,1)]
```

Sample Output

```
Closest Pair: ((1, 2), (3, 1))
```

```
Distance = 2.23606797749979
```

Result

Thus, the closest pair of points was found successfully using the brute-force approach.

Experiment 1: Trophy Ranking System

Aim

To extract the top 5 highest scores from a list using the Selection Sort technique without sorting the entire list.

Algorithm

1. Read the list of scores.
2. Repeat for the first **k** positions:
 - Find the maximum element in the remaining list.
 - Swap it with the current position.
3. Print the first **k** elements as the top scores.

Program

```
def top_k_scores(scores, k):  
    arr = scores[:]  
    n = len(arr)  
  
    for i in range(min(k, n)):  
        max_index = i  
        for j in range(i + 1, n):  
            if arr[j] > arr[max_index]:  
                max_index = j  
        arr[i], arr[max_index] = arr[max_index], arr[i]  
  
    return arr[:k]  
  
scores = [72, 88, 65, 90, 77, 95, 60, 83, 91, 68]  
k = 5  
print("Top Scores:", top_k_scores(scores, k))
```

Sample Input

Scores = [72,88,65,90,77,95,60,83,91,68]

k = 5

Sample Output

Top Scores: [95, 91, 90, 88, 83]

Result

Thus, the top 5 scores were extracted successfully using Selection Sort.

Experiment 2: Memory-Constrained Embedded Device**Aim**

To sort temperature readings using Selection Sort while minimizing the number of swaps.

Algorithm

1. Read the temperature readings.
2. Find the minimum element in the unsorted portion.
3. Swap only if necessary.
4. Count the number of swaps.
5. Display the sorted array and total swaps.

Program

```
def selection_sort_min_writes(arr):
```

```
    arr = arr[:]
```

```
    swaps = 0
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_index = i
```

```
        for j in range(i + 1, n):
```



```

        if arr[j] < arr[min_index]:
            min_index = j

    if min_index != i:
        arr[i], arr[min_index] = arr[min_index], arr[i]
        swaps += 1

return arr, swaps

temps = [23.5, 19.2, 25.1, 18.8, 21.4]
result, swaps = selection_sort_min_writes(temps)
print("Sorted:", result)
print("Swaps:", swaps)

```

Sample Input

[23.5,19.2,25.1,18.8,21.4]

Sample Output

Sorted: [18.8, 19.2, 21.4, 23.5, 25.1]

Swaps: 3

Result

Thus, the array was sorted successfully with minimum write operations.

Experiment 3: Library Book Reordering

Aim

To arrange books in ascending order using Selection Sort and count the physical moves.

Algorithm

1. Read the book IDs.

2. Find the smallest ID.
3. Swap it into its correct position.
4. Count each swap.
5. Display the sorted IDs and total moves.

Program

```
def reorder_shelf(arr):  
    arr = arr[:]   
    moves = 0  
    n = len(arr)  
  
    for i in range(n):  
        min_index = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_index]:  
                min_index = j  
  
        if min_index != i:  
            arr[i], arr[min_index] = arr[min_index], arr[i]  
            moves += 1  
  
    return arr, moves  
  
books = [305, 102, 250, 118, 199, 400, 101]  
ordered, moves = reorder_shelf(books)  
  
print("Ordered Books:", ordered)
```

```
print("Moves:", moves)
```

Sample Input

```
[305,102,250,118,199,400,101]
```

Sample Output

Ordered Books:

```
[101,102,118,199,250,305,400]
```

Moves: 5

Result

Thus, the books were reordered successfully using Selection Sort.

Experiment 4: Contest Prize Distribution

Aim

To rank contest participants based on their scores using Selection Sort in descending order.

Algorithm

1. Read participant names and scores.
2. Find the participant with the highest score.
3. Swap into the current position.
4. Repeat until all participants are ranked.
5. Display the ranking.

Program

```
def distribute_prizes(data):
```

```
    arr = data[:]
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        max_index = i
```

```
for j in range(i + 1, n):  
    if arr[j][1] > arr[max_index][1]:  
        max_index = j  
  
arr[i], arr[max_index] = arr[max_index], arr[i]
```

```
return arr
```

```
participants = [('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)]
```

```
ranking = distribute_prizes(participants)
```

```
for item in ranking:
```

```
    print(item)
```

Sample Input

```
('Asha',88)
```

```
('Ravi',95)
```

```
('Meera',79)
```

```
('Dev',95)
```

Sample Output

```
('Ravi',95)
```

```
('Dev',95)
```

```
('Asha',88)
```

```
('Meera',79)
```

Result

Thus, the participants were ranked successfully using Selection Sort.

Experiment 5: Small Dataset in a Mobile App

Aim

To sort a small list of product prices using Selection Sort.

Algorithm

1. Read the list of prices.
2. Find the minimum element.
3. Swap it with the current position.
4. Repeat until the list is sorted.
5. Display the sorted prices.

Program

```
def selection_sort(arr):  
    arr = arr[:]   
    n = len(arr)  
  
    for i in range(n):  
        min_index = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_index]:  
                min_index = j  
  
        arr[i], arr[min_index] = arr[min_index], arr[i]  
  
    return arr  
  
prices = [499,129,899,45,275,60,310,150]
```

```
print("Sorted Prices:", selection_sort(prices))
```

Sample Input

```
[499,129,899,45,275,60,310,150]
```

Sample Output

Sorted Prices:

```
[45,60,129,150,275,310,499,899]
```

Result

Thus, the product prices were sorted successfully using Selection Sort.

Experiment 20: Exam Result Slip Correction**Aim**

To sort a nearly sorted list of student roll numbers using Optimized Bubble Sort.

Algorithm

1. Read the list of roll numbers.
2. Compare adjacent elements.
3. Swap if they are in the wrong order.
4. Use a flag to check whether any swap occurs.
5. If no swap occurs, stop the algorithm.
6. Display the sorted list and the number of passes.

Program

```
def optimized_bubble_sort(arr):
```

```
    arr = arr[:]
```

```
    n = len(arr)
```

```
    passes = 0
```

```
for i in range(n):
    swapped = False
    passes += 1
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True
    if not swapped:
        break
return arr, passes
```

```
rolls = [101,102,104,103,105,107,106,108]
result, passes = optimized_bubble_sort(rolls)
print("Sorted:", result)
print("Passes:", passes)
```

Sample Input

```
[101,102,104,103,105,107,106,108]
```

Sample Output

```
Sorted: [101,102,103,104,105,106,107,108]
```

```
Passes: 3
```

Result

Thus, the roll numbers were sorted successfully using Optimized Bubble Sort.

Experiment 21: Traffic Signal Priority Queue

Aim

To arrange vehicles according to their priority using Bubble Sort.

Algorithm

1. Assign priorities to vehicles.
2. Compare adjacent vehicles.
3. Swap according to priority.
4. Repeat until the queue is sorted.
5. Display the priority queue.

Program

```
def bubble_sort_queue(queue):  
    priority = {"ambulance":3, "bus":2, "car":1}  
    arr = queue[:]   
    n = len(arr)  
  
    for i in range(n):  
        for j in range(n-i-1):  
            if priority[arr[j]] < priority[arr[j+1]]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
  
    return arr
```

```
queue = ['car','car','bus']  
queue.append('ambulance')
```

```
print(bubble_sort_queue(queue))
```

Sample Input

```
['car','car','bus','ambulance']
```

Sample Output

```
['ambulance','bus','car','car']
```


Result

Thus, the vehicles were arranged successfully based on priority using Bubble Sort.

Experiment 22: Bubble Sort Visualization Tool

Aim

To display the array after every pass of Bubble Sort.

Algorithm

1. Read the array.
2. Perform Bubble Sort.
3. After every pass, store the current array.
4. Display each pass.
5. Display the final sorted array.

Program

```
def bubble_sort_with_frames(arr):  
    arr = arr[:]   
    frames = [arr[:]]  
    n = len(arr)  
  
    for i in range(n):  
        for j in range(n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
            frames.append(arr[:])  
    return frames  
  
frames = bubble_sort_with_frames([5,1,4,2,8])
```

```
for i, frame in enumerate(frames):
```

```
    print("Pass", i, ":", frame)
```

Sample Input

[5,1,4,2,8]

Sample Output

Pass 0 : [5,1,4,2,8]

Pass 1 : [1,4,2,5,8]

Pass 2 : [1,2,4,5,8]

Pass 3 : [1,2,4,5,8]

Pass 4 : [1,2,4,5,8]

Pass 5 : [1,2,4,5,8]

Result

Thus, the Bubble Sort process was visualized successfully.

Experiment 23: Sorting Sensor Alerts by Severity

Aim

To compare normal Bubble Sort and Optimized Bubble Sort.

Algorithm

1. Read the alert levels.
2. Perform normal Bubble Sort.
3. Perform optimized Bubble Sort.
4. Compare the number of comparisons.
5. Display the sorted alerts.

Program

```
def bubble_sort_plain(arr):
```

```
arr = arr[:]
comp = 0
n = len(arr)

for i in range(n):
    for j in range(n-i-1):
        comp += 1
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr, comp
```

```
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
```

```
result, comparisons = bubble_sort_plain(alerts)
```

```
print(result)
```

```
print("Comparisons:", comparisons)
```

Sample Input

```
[2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
```

Sample Output

```
[1,1,1,1,2,2,2,2,2,3,3,3,4,4,5]
```

```
Comparisons: 105
```

Result

Thus, the alerts were sorted successfully using Bubble Sort.

Experiment 24: Card Game Hand Sorting

Aim

To sort a player's cards using Optimized Bubble Sort.

Algorithm

1. Read the cards.
2. Insert the new card.
3. Compare adjacent cards.
4. Swap if required.
5. Display the sorted hand.

Program

```
def bubble_sort_hand(arr):  
  
    arr = arr[:]   
  
    passes = 0  
  
    n = len(arr)  
  
    for i in range(n):  
  
        swapped = False  
  
        passes += 1  
  
        for j in range(n-i-1):  
  
            if arr[j] > arr[j+1]:  
  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
  
                swapped = True  
  
        if not swapped:  
  
            break  
  
    return arr, passes
```

```
hand = [2,4,6,8,9,11,13]
```

```
hand.append(7)
```

```
sorted_hand, passes = bubble_sort_hand(hand)
```

```
print("Sorted Hand:", sorted_hand)
```

```
print("Passes:", passes)
```

Sample Input

```
[2,4,6,8,9,11,13,7]
```

Sample Output

Sorted Hand:

```
[2,4,6,7,8,9,11,13]
```

Passes: 2

Result

Thus, the player's hand was sorted successfully using Optimized Bubble Sort.

Experiment 26: Live Leaderboard Updates

Aim

To insert an updated player's score into an already sorted leaderboard using Insertion Sort.

Algorithm

1. Read the sorted leaderboard.
2. Insert the updated score at the end.
3. Shift larger scores to the right.
4. Place the updated score in its correct position.

5. Display the updated leaderboard.

Program

```
def insert_updated_score(board, score):  
    arr = board[:]  
    arr.append(score)  
    shifts = 0  
  
    i = len(arr) - 2  
    while i >= 0 and arr[i] < score:  
        arr[i + 1] = arr[i]  
        shifts += 1  
        i -= 1  
  
    arr[i + 1] = score  
    return arr, shifts  
  
board = [980, 875, 760, 690, 500]  
  
updated, shifts = insert_updated_score(board, 820)  
  
print("Updated Leaderboard:", updated)  
print("Shifts:", shifts)
```

Sample Input

[980,875,760,690,500]

820

Sample Output

Updated Leaderboard:

[980,875,820,760,690,500]

Shifts: 3

Result

Thus, the updated score was inserted successfully using Insertion Sort.

Experiment 27: Card Sorting While Playing

Aim

To arrange playing cards one by one using Insertion Sort.

Algorithm

1. Start with an empty hand.
2. Insert each new card into its correct position.
3. Shift larger cards to the right.
4. Repeat for all cards.
5. Display the sorted hand.

Program

```
def pick_up_card(hand, card):  
    hand.append(card)  
  
    i = len(hand) - 2  
    while i >= 0 and hand[i] > card:  
        hand[i + 1] = hand[i]  
        i -= 1  
  
    hand[i + 1] = card  
    return hand
```

```
hand = []
```

```
for card in [7,2,9,4,1]:
```

```
    hand = pick_up_card(hand, card)
```

```
print(hand)
```

Sample Input

7 2 9 4 1

Sample Output

[1,2,4,7,9]

Result

Thus, the cards were arranged successfully using Insertion Sort.

Experiment 28: Real-Time Stock Price Feed

Aim

To maintain stock prices in sorted order using Insertion Sort.

Algorithm

1. Read stock prices one by one.
2. Insert each price into its proper position.
3. Shift larger values to the right.
4. Repeat until all prices are inserted.
5. Display the sorted list.

Program

```
def insert_price(prices, price):
```

```
    prices.append(price)
```



```

i = len(prices) - 2
while i >= 0 and prices[i] > price:
    prices[i + 1] = prices[i]
    i -= 1

prices[i + 1] = price
return prices

prices = []

for p in [102.5,98.3,105.1,100.0,97.8]:
    prices = insert_price(prices, p)

print(prices)

```

Sample Input

102.5 98.3 105.1 100.0 97.8

Sample Output

[97.8,98.3,100.0,102.5,105.1]

Result

Thus, the stock prices were maintained successfully using Insertion Sort.

Experiment 29: Playlist Reordering by Recently Added

Aim

To insert a newly added song into a sorted playlist using Insertion Sort.

Algorithm

1. Read the playlist.
2. Append the new song.
3. Shift songs with greater duration.
4. Insert the new song at the correct position.
5. Display the updated playlist.

Program

```
def insert_song(playlist, song):  
    playlist.append(song)  
  
    i = len(playlist) - 2  
    while i >= 0 and playlist[i][1] > song[1]:  
        playlist[i + 1] = playlist[i]  
        i -= 1  
  
    playlist[i + 1] = song  
    return playlist  
  
playlist = [('Intro',120),('Chill Beat',210),('Long Jam',340)]  
  
updated = insert_song(playlist, ('Quick Track',180))  
  
print(updated)  
  
Sample Input  
('Intro',120)  
('Chill Beat',210)  
('Long Jam',340)
```

('Quick Track',180)

Sample Output

[('Intro',120),

('Quick Track',180),

('Chill Beat',210),

('Long Jam',340)]

Result

Thus, the playlist was reordered successfully using Insertion Sort.

Experiment 30: Nearly Sorted Sensor Log Correction

Aim

To sort a nearly sorted sensor log using Insertion Sort and count the shifts.

Algorithm

1. Read the sensor log.
2. Perform Insertion Sort.
3. Count the number of shifts.
4. Display the sorted log.
5. Display the total shifts.

Program

```
def insertion_sort_count_shifts(arr):
```

```
    arr = arr[:]
```

```
    shifts = 0
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
while j >= 0 and arr[j] > key:
```

```
    arr[j + 1] = arr[j]
```

```
    shifts += 1
```

```
    j -= 1
```

```
arr[j + 1] = key
```

```
return arr, shifts
```

```
log = [18.2,18.5,18.9,17.9,19.1,19.4,19.0]
```

```
sorted_log, shifts = insertion_sort_count_shifts(log)
```

```
print("Sorted Log:", sorted_log)
```

```
print("Shifts:", shifts)
```

Sample Input

```
[18.2,18.5,18.9,17.9,19.1,19.4,19.0]
```

Sample Output

```
Sorted Log:
```

```
[17.9,18.2,18.5,18.9,19.0,19.1,19.4]
```

```
Shifts: 4
```

Result

Thus, the nearly sorted sensor log was corrected successfully using Insertion Sort. These experiments correspond to the Insertion Sort section of the uploaded lab sheet.